LUIS ROMERO

AI Engineer | Backend Developer (Node.js/TypeScript/Python)

Mérida, Venezuela | +58 424 709 2980 | lueduar15@gmail.com | linkedin.com/in/hernandezrs955 | github.com/ROMEROLUIS15

RESUMEN PROFESIONAL

AI Engineer y Backend Developer que lleva productos de IA a producción y los sostiene ahí. Construyo backend para una empresa de software de Texas (Estados Unidos), donde sostengo la entrega del producto con desarrollo asistido por IA bajo mi diseño y revisión: 1.050+ pull requests integrados en tres meses. En paralelo construyo y opero sistemas de IA con usuarios reales todos los días: agentes de WhatsApp, voz y llamadas que agendan citas de forma autónoma, un asistente de taller en Python/LangGraph que atiende a clientes y a la administración, y un asistente industrial que responde consultando el historial real de servicio. Mi trabajo no es integrar un LLM: es diseñar los controles que impiden que invente datos o actúe sobre el registro equivocado, y medir su fiabilidad con evaluaciones automáticas en vez de suponerla.

EXPERIENCIA

Backend Developer | Complexity (Texas, Estados Unidos, Remoto) Julio 2026 - Presente

Proyecto Factory: backend de una plataforma B2B de directorio de empresas y de su panel de administración, con app móvil. API REST, tiempo real por WebSockets, colas de trabajo, búsqueda full-text y agentes de IA (Node.js, NestJS, TypeScript, PostgreSQL, Redis/BullMQ, Elasticsearch, AWS).

- Audité el backend al llegar, presenté los hallazgos y me aprobaron el plan de remediación, que dirigí hasta cerrarlo sin frenar el roadmap: con refactoring incremental eliminé el acoplamiento indebido entre módulos, bajé las dependencias circulares de 31 a 0 y llevé la lógica de negocio a 330 casos de uso sobre arquitectura hexagonal.
- Diseñé el harness de agentes de IA con el que trabajo: reglas numeradas que el propio código cita, gates de arquitectura y calidad, y un control que pide evidencia de cada afirmación del agente (Orca, Gentle AI, MCP, Spec-Driven Development). Con él planifiqué el trabajo en 700+ tickets y entregué 1.050+ pull requests integrados y 20+ releases a producción en tres meses, revisando y aprobando cada cambio.
- Construí desde cero el pipeline de calidad y seguridad. Al llegar, ningún pull request ejecutaba pruebas. Hoy cada cambio pasa por tipado estricto, pruebas unitarias, de integración y end-to-end, escaneo de seguridad (SAST/SCA, secretos) y reglas de arquitectura, y un PR completo se valida en ~6 min. Las pruebas unitarias pasaron de 20 a 1.000+ suites. El chequeo de calidad bajó de 9,4 a 3,8 min, con ejecución en paralelo y suites de integración elegidas por grafo de dependencias.
- Lideré de punta a punta el backend de "Reclama tu empresa", el programa de salida al mercado (GTM), hoy en producción. Lo alimenta la carga asistida por IA que llevé a producción: un agente arma perfiles de empresa y borradores de producto, y nada se publica sin aprobación humana (worker BullMQ, flujo de aceptar o rechazar).
- Diseñé y construí un motor de extracción con IA con prácticas actuales de ingeniería de agentes: cada dato lleva la cita de su fuente, dos modelos deben coincidir y el código, no el modelo, decide qué se guarda; la calidad se mide con evals contra la decisión del revisor humano y trazas en Langfuse. Casi triplicó la precisión al clasificar categorías.
- Llevé la seguridad a nivel de fila (RLS) de PostgreSQL a producción (63 tablas, con pruebas pgTAP que bloquean el merge), resolví en menos de 4 h un reporte urgente de exposición de datos personales y operé despliegues en AWS, con dos incidentes de producción cerrados en menos de una hora.

IBIME Connect (Plataforma institucional, Remoto) Enero 2025 - Julio 2026

Coordinador de Desarrollo de Software (Febrero 2026 - Julio 2026) | Backend / Full-Stack Developer (Enero 2025 - Febrero 2026)

- Construí el asistente de IA con el que una red pública de bibliotecas atiende las consultas de los ciudadanos, dentro de la plataforma que reemplazó su sistema antiguo. La institución la aprobó, la adoptó como su plataforma oficial y, a raíz de esa entrega, me designó Coordinador de Desarrollo de Software.
- Conseguí que el asistente no inventara respuestas manteniendo al modelo fuera del flujo de decisión: un clasificador de intención por reglas (sin LLM) enruta la conversación, y si el correo del ciudadano ya existe la respuesta sale de la base de datos sin llamar al modelo —menos coste, menos latencia y ninguna oportunidad de alucinar en esas consultas—. Una cadena determinista (intención, protección de datos, umbral de coincidencia, revisión y política de salida) valida antes de responder.
- Implementé búsqueda semántica sobre los documentos de la institución: RAG con PostgreSQL + pgvector y embeddings de Gemini de 768 dimensiones, con inferencia en Groq gpt-oss-20b configurable por entorno y caché de embeddings en Redis, y un umbral de coincidencia que prefiere no responder antes que responder mal.
- Construí un agente LangGraph con estado (extractor, validador y corrector, con reintentos acotados) que lee los PDFs del catálogo institucional y los convierte en datos estructurados y validados, descartando duplicados por contenido.
- Endurecí la plataforma para tratar datos personales de ciudadanos: bloqueé la escritura anónima sobre las tablas sensibles (RLS), evité inscripciones duplicadas y definí cuánto tiempo se conserva cada dato y cómo se borra. Arquitectura hexagonal con inyección de dependencias, logs estructurados y captura de errores (Sentry); 420+ tests en CI.

Backend Developer | Kiura (Barranquilla, Colombia, Remoto) Abril 2024 - Diciembre 2024

- Construí los servicios backend de una plataforma de teleterapia: pagos por suscripción que activan o suspenden el acceso del paciente (MercadoPago vía webhook), chat en tiempo real (Socket.IO) y videosesiones con acceso firmado por sala (Agora), sobre un dominio de 26 entidades con doble autenticación (JWT + Google OAuth) y permisos por rol.

PROYECTOS DE IA

Fundador y AI Engineer | Cronix (en producción; operación y mantenimiento) Enero 2026 - Presente

SaaS multi-tenant para negocios con citas, construido y operado solo. En producción 24/7: 170+ citas reales gestionadas por los agentes y recordatorios automáticos, sin intervención manual. 1.600+ tests (Vitest, Playwright, pgTAP) y CI/CD con Sentry.

- Construí los agentes de WhatsApp y voz que agendan, reagendan y cancelan citas solos: el cliente lo pide hablando, a cualquier hora, y nadie del negocio tiene que intervenir. Motor de orquestación propio (sin framework), con rutas deterministas que resuelven la mayoría de los casos sin llamar al modelo, y acceso restringido a herramientas (Groq + Deepgram, 12 capacidades en el agente de voz).
- Construí un agente de llamadas en tiempo real que atiende por WhatsApp y agenda citas en Cronix hablando (Python, Pipecat Flows, WebRTC, Deepgram): el modelo conversa y el código decide cada dato y cada escritura, incluida la confirmación del cliente. Del lado de Cronix, una API firmada (HMAC) con idempotencia y un bloqueo que impide reservar dos veces el mismo horario.
- Elegí el modelo del agente de llamadas midiendo, no por intuición: en el dataset golden obtuvo 1,00 en no inventar datos y en no escribir sin pedido, el primer token llega en 391 ms (p50) y cada llamada queda trazada en Sentry con su latencia y su costo.
- Evité que el agente inventara datos o actuara sobre el cliente equivocado —el riesgo real de darle a un LLM permiso de escritura sobre una base de datos con clientes reales— con 6 capas de control: sacar al modelo del camino cuando la ruta es determinista, restringir qué herramientas puede llamar y con qué argumentos, anclar cada dato al texto que el usuario dijo de verdad, resolver nombres con umbral de confianza, obligar a confirmar en dos pasos antes de escribir, y un segundo modelo que revisa la operación y puede vetarla (LLM-as-judge).
- Construí el sistema que comprueba si el agente responde bien: guardé conversaciones reales como referencia y las vuelvo a ejecutar en cada cambio; el CI bloquea el merge si el agente empeora (conversaciones golden), y aparte lo mido con DeepEval y un juez LLM (relevancia y alucinación).
- Monté la observabilidad del agente: cada turno queda registrado con su resultado, tokens, latencia (p50/p95) y errores, visible por negocio en un panel propio sobre PostgreSQL, con envío en paralelo a LangSmith y alertas de error. Todo corre sobre 9 Edge Functions (Deno).
- Audité mi propia seguridad hasta encontrarle un agujero: descubrí que 13 funciones de la base de datos permitían ver datos de otros negocios, reproduje el ataque para confirmarlo, lo cerré con una validación de pertenencia y lo dejé cubierto por 147 pruebas automáticas de seguridad (pgTAP sobre 59 políticas de aislamiento).

AI Engineer / Backend | Motosmax Cordialidad (Cliente, Remoto) (entregado por fases; en producción con soporte) Junio 2026 - Presente

- Automaticé la atención al cliente y a la administración de un taller de motos con un microservicio en Python (FastAPI + LangGraph): un agente atiende por WhatsApp a los clientes y otro asiste a la administración con memoria de sesión, herramientas propias, reportes periódicos y aviso de stock bajo. El modelo (Groq/DeepSeek, con reserva ante fallo) es configurable por entorno, y el canal de WhatsApp maneja la ventana de 24h de Meta y reporta los fallos de envío.
- Lo conecté al sistema real del taller sin darle acceso suelto: el microservicio de IA habla con la API (NestJS, arquitectura hexagonal, 37 modelos de datos) mediante credenciales que caducan a los 5 minutos, y los datos sensibles van cifrados. 400+ tests (Jest, pytest, Playwright) con lint y tipado estricto en CI; cada cambio arranca de una especificación versionada (Spec-Driven Development).

AI Engineer / Backend | CMMS Hidrobombas Mérida (Cliente, Remoto) (entregado por fases; en producción con soporte) Abril 2026 - Presente

- Construí el asistente de IA de una empresa de mantenimiento industrial: responde preguntas consultando el historial real de reportes en vez de improvisar (LangGraph + RAG persistente sobre pgvector), con orquestación multi-grafo —un agente con herramientas y un grafo de diagnóstico— y la capa de IA aislada tras puertos para poder probarla sin depender del proveedor de turno (almacén vectorial en memoria en desarrollo, pgvector en producción). 394 tests, 82% de cobertura en backend.

Advanced Product Search API (Reto técnico con el que entré a Complexity; desplegado y público) Julio 2026

En vivo: advanced-search-api-chet.onrender.com/docs · github.com/ROMEROLUIS15/advanced-search-api

- Construí el buscador que ordena los resultados por lo que la persona realmente busca y no por orden de llegada: relevancia BM25 sobre Elasticsearch 8 con tolerancia a errores de tipeo, ajustada por popularidad y por qué tan reciente es el producto, con facetas cuyos contadores excluyen su propio filtro (para poder ampliar la búsqueda en vez de quedarse sin resultados), autocompletado y sugerencias "quisiste decir".
- Lo entregué desplegado y verificado: 500+ pruebas (97% de cobertura, incluyendo integración contra Elasticsearch real), carga k6 (398.000 peticiones, 0 fallos) y seguridad automatizada en CI sobre código, dependencias, secretos del historial, imagen desplegada y API en ejecución (CodeQL, gitleaks, Trivy + SBOM, OWASP ZAP).

HABILIDADES TÉCNICAS

IA y LLM: LLM Integration, AI Agents, Multi-Agent Orchestration, LangGraph, LangChain, Tool Calling / Function Calling, Structured Outputs, Agentic AI, Vercel AI SDK, OpenAI API, RAG (Retrieval-Augmented Generation), Semantic Search, Embeddings, Vector Databases (pgvector), Prompt Engineering, Anti-Hallucination, AI Guardrails, LLM-as-a-Judge, LLM Evaluation (DeepEval, conversaciones golden), LLM Observability (LangSmith, Langfuse, trazas, tokens, latencia), MCP (Model Context Protocol), Voice AI, Pipecat, Pipecat Flows, WebRTC, Speech-to-Text, Text-to-Speech, OpenRouter (STT/TTS), Groq, DeepSeek, Gemini, Deepgram, n8n

Harness Engineering y desarrollo asistido por IA: Claude Code (Anthropic), Codex (OpenAI), Orca (ADE), Gentle AI, Engram (memoria persistente de agentes), MCP (Linear, Sentry, AWS, Resend, Figma), orquestación multi-agente, subagentes, hooks y skills para agentes, Context Engineering

Lenguajes y frameworks: TypeScript, JavaScript, Node.js, NestJS, Python, FastAPI, Pydantic, asyncio, uv, Express, Deno, Next.js, React

Backend y datos: REST APIs, OpenAPI/Swagger, Email transaccional (Resend, React Email), Socket.IO, Microservicios, Serverless / Edge Functions, Event-Driven Architecture, Eventos de dominio, Unit of Work, Webhooks, Colas (BullMQ, QStash), Idempotencia, Arquitectura Hexagonal, Clean Architecture, Architecture Decision Records (ADRs), SOLID, Refactoring, Deuda técnica, Inyección de Dependencias, PostgreSQL, SQL, pgvector, Redis, Prisma, TypeORM, Supabase, Neon, Elasticsearch, MySQL, Row-Level Security, Multi-tenant

DevOps, pruebas y seguridad: Docker, AWS (S3, EC2 Auto Scaling, ECR, CloudWatch, Secrets Manager), Git, GitHub Actions, Husky, Linear, CI/CD (Continuous Integration / Continuous Deployment), Vercel, Render, Cloudflare (R2), Sentry, OpenTelemetry, Observabilidad, Load Testing (k6), pytest, Hypothesis (property-based testing), Snapshot testing, mypy (strict), Ruff, Bandit, pip-audit, Vitest, Jest, Playwright, pgTAP, SAST/DAST/SCA (CodeQL, Semgrep, OWASP ZAP, Trivy, gitleaks, Dependabot), JWT, OAuth 2.0, RBAC, Rate Limiting, HMAC, Threat Modeling, Auditoría de seguridad, Spec-Driven Development (SDD)

EDUCACIÓN

Ingeniería en Sistemas Informáticos | UPTM Kléber Ramírez - Graduado: Oct 2026

Bootcamp Full-Stack | Academlo - Mar 2024

IDIOMAS

Español: Nativo | Inglés: Intermedio (B1+), lectura y escritura técnica avanzada